Motion Profiling in Vex

Cooper Brosgall

2D Motion Profiling is becoming much more popular in the context of Vex Robotics. This document will not hand you the code for creating a motion profile, but by following through it, it will help you understand both how it works, and how you can go about implementing it.

September 03, 2025

Contents

1. Introduction

1.1. Background

To best understand this document, you should have some level of understanding of the following:

- Vectors in 2D space: e.g. $\begin{bmatrix} x \\ y \end{bmatrix}$
- Cartesian (rectangular) coordinates in 2D space
- 2D kinematics (position, velocity, acceleration, etc.)

1.2. Motivation

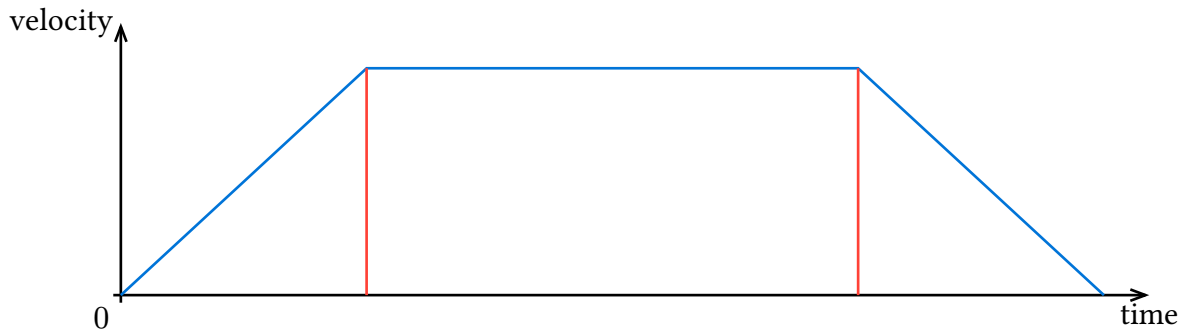
When I started competing in Vex, Motion Profiling was nearly unheard of, and there were very few documents explaining how it worked, let alone how to implement it effectively. Having spent the time learning how to implement it, and seeing it being popularized by teams such as 2654Echo, I wanted to share this knowledge with the rest of the Vex community so that more people could understand it and push it further.

1.3. Units

For the purposes of this document, I will be working primarily with inches / imperial units, as the Vex field is defined using them. You can easily change this to use meters, and this can be useful for some algorithms such as Ramsete[1]. All angles will be defined in radians, as that is the default unit system for trigonometric functions such as $\sin()$ and $\cos()$

2. 1D Motion Profiling

The first thing you need to understand to implement a 2D motion profile is what a motion profile is at all. In this document, I will be covering primarily trapezoidal motion profiles, which are constant acceleration profiles, and thus when plotted as velocity vs time, look like a trapezoid:



Profiles like this are relatively simple to implement, as they can be done purely with logic and knowledge of the basic kinematic formulas.

In the case of a 1D motion profile, it is easy to calculate the profile of v(t) or v(x), meaning with respect to either the current time, or current distance. Either of these options can be chosen to be followed. This will change when we get to the 2D profile, which can only be generated with respect to distance, at least using the method I will describe. For this profile, I will describe the equations for either method.

For the acceleration phase of the profile, the time needed to complete it is $t_{\text{accel}} = \frac{v_{\text{max}}}{a_{\text{max}}}$, and the distance $d_{\text{accel}} = \frac{v_{\text{max}}^2}{2a_{\text{max}}}$.

For the coasting phase, you use distance spent accelerating and decelerating, which are the same, and subtract it from the total distance. This looks like this:

$$d_{\text{coast}} = d_{\text{total}} - 2d_{\text{accel}}$$

$$t_{\text{coast}} = \frac{d_{\text{coast}}}{v_{\text{max}}}$$

As mentioned, the time and distance for deceleration will be the same as acceleration, as this profile is symmetrical. These formulas can be adjusted to allow for asymmetrical acceleration and deceleration rates, but I'll leave that as an exercise to implement yourself.

Based on these formulas, you can derive the following piecewise functions:

$$v(t) := \begin{cases} a_{\text{max}} * t, & 0 \le t < t_{\text{accel}} \\ v_{\text{max}}, & t_{\text{accel}} \le t < t_{\text{accel}} + t_{\text{coast}} \\ v_{\text{max}} - a_{\text{max}} * (t - t_{\text{accel}} - t_{\text{coast}}), & t_{\text{accel}} + t_{\text{coast}} \le t \le t_{\text{total}} \end{cases} \tag{1}$$

$$v(d) := \begin{cases} \sqrt{2 * a_{\text{max}} * d}, & 0 \le d < d_{\text{accel}} \\ v_{\text{max}}, & d_{\text{accel}} \le d < d_{\text{accel}} + d_{\text{coast}} \\ \sqrt{2 * a_{\text{max}} * (d_{\text{total}} - d)}, & d_{\text{accel}} + d_{\text{coast}} \le d \le d_{\text{total}} \end{cases} \tag{2}$$

3. 2D Motion Profiling

In this document, I will be covering a technique commonly known as double-pass motion profiling, which is quite robust to arbitrary kinematic constraints. As mentioned, this will be covering trapezoidal, or constant-acceleration, profiles, but it can be extended to constant-jerk with some effort.

3.1. Path Generation

The first step of 2D motion profiling is generating a path to follow. The requirements for such a path are relatively simple. For this method, assuming you are using a differential drive (tank drive), they will need to be at least doubly differentiable, and they should be in the form of a parametric function of t ($\begin{bmatrix} x \\ y \end{bmatrix} = f(t)$).

In this paper, I will be using cubic Beziers, but you can easily use any other similar curve, such as Hermite splines, etc.

Bezier curves are generally defined in the form of:

$$f(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3 \tag{3}$$

However, for this paper, I will use the matrix form, as it can often be computed much more efficiently. This matrix form of a cubic bezier[2] looks like this:

$$f(t) = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix} \tag{4}$$

There are similar matrix formulations for the first and second derivatives of a cubic bezier, which we will also find useful as a part of trajectory generation, and are as follows:

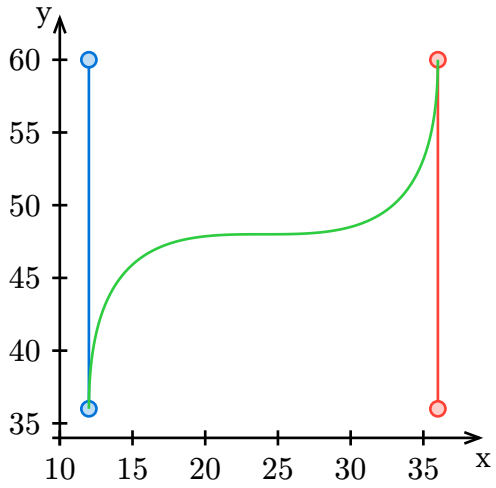
$$f'(t) = \begin{bmatrix} t^2 & t & 1 \end{bmatrix} \begin{bmatrix} -3 & 9 & -9 & 3 \\ 6 & -12 & 6 & 0 \\ -3 & 3 & 0 & 0 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix} \tag{5}$$

$$f''(t) = \begin{bmatrix} t & 1 \end{bmatrix} \begin{bmatrix} -6 & 18 & -18 & 6 \\ 6 & -12 & 6 & 0 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix} \tag{6}$$

For example, if we want to generate a path from (12, 36) to (36, 48), with control points (12, 48) and (36, 36), we will need to do the following calculation:

$$\begin{bmatrix} x & y \end{bmatrix} = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 12 & 36 \\ 12 & 60 \\ 36 & 36 \\ 36 & 60 \end{bmatrix} \tag{7}$$

by running this calculation over a range of $t \in [0, 1]$, we can generate the following curve:



3.2. Trajectory Generation

As mentioned earlier, this method of trajectory generation in 2D motion profiling involves two passes, and also requires that we parametrize the path function by distance.

The two passes are mostly identical, with one being from start to end, and one in reverse. The core idea of the passes are a loop over each point, that follows a series of specified constraints. This will include a maximum acceleration, but in the case of a differential drive, will also include limits on linear speed based on angular speed, and can even include additional constraints to prevent side-to-side slipping.

From here on, I will focus on differential drives, but much of this can be applied to any generic kinematic constraints.

3.2.1. Point Generation

The first issue we run into is converting the bezier curve into a set of evenly spaced points, with spacing Δd (I suggest 0.1in). There are a few ways to do this, but the easiest and one of the most efficient methods is based on using the derivative of the path. Because the derivative of the bezier $f(t)$ is equal to $\frac{\mathrm{dx}}{\mathrm{dt}}$, we can calculate the proper increase in t by taking

$$\Delta t = \frac{\Delta d}{f'(t)} \tag{8}$$

3.2.2. Kinematic Constraints

Now that we can generate evenly spaced points, we need to calculate velocity we can get to from each point. The idea of having two passes is that both passes are trying to accelerate as much as they can, and the final profile is made up of the minimum of the two.

For a given pass, we know the change is distance is Δd, so we can calculate what the velocity change will be based on our current velocity and max acceleration using:

$$v_f = \sqrt{v^2 + 2 * a_{\max} * \Delta d} \tag{9}$$

3.2.3. Differential Drive Kinematic Constraints

The next constraint is specific to differential drives, and it is because the robot linear and angular speeds have to be traded off between each other. This means when we take a sharp turn, the robot can't go as fast. To derive the maximum speed at a given turn radius, we can look at the kinematic equations for a tank drive.

For these equations, you will need the following definitions:
- l is the track width of the robot, which is the distance between wheels
- ω is the angular velocity of the robot
- $v_{\max}$ is the maximum linear speed of the robot
- v_l and v_r are the velocities of the left and right wheels respectively
- r is the radius of the current turn/path
- κ is the curvature of the current turn/path, and is equal to $\frac{1}{r}$

The equations we will need are

$$v = \omega R \tag{10}$$

$$v_r = \omega\left(r + \frac{l}{2}\right) \tag{11}$$

$$v_l = \omega\left(r - \frac{l}{2}\right) \tag{12}$$

By assuming that our outer wheel is at the maximum speed, and therefore is the limiting factor, we can find what linear robot velocity would cause this at a given radius. With $v_r = v_{\max}$, and Equation 11 we can say $\omega = \frac{v_{\max}}{r+\frac{l}{2}}$, and combining that with Equation 10 we can show $v = \frac{rv_{\max}}{r+\frac{l}{2}}$. By simplifying and solving this in terms of curvature, we can finally show:

$$v = \frac{2v_{\max}}{2 + l\kappa} \tag{13}$$

3.2.4. Forward and Backwards Pass

We now have almost all the knowledge we need to actually implement the forward and backwards passes. One further equation we will likely want is the formula for curvature, which is:

$$\kappa = \frac{x'y'' - x''y'}{(x'^2 + y'^2)^{\frac{3}{2}}} \tag{14}$$

where x' denote the first derivative of the x position of our path, and y'' denotes the second derivative of the y position.

With that in mind, here are the steps each pass should complete as it loops:

1. Get the current position, its derivative, and its second derivative.
2. Calculate the curvature based on Equation 14.
3. Calculate the max speed due to curvature based on Equation 13.

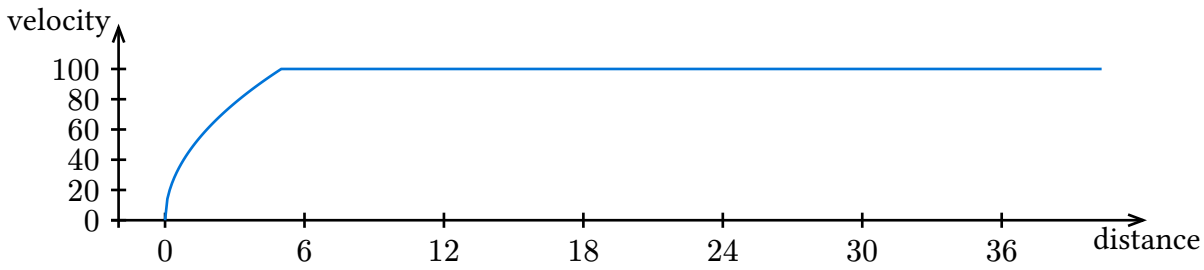
4. Calculate the maximum speed you can accelerate to using Equation 9 and the previous speed.
5. Take the minimum of these values, along with your overall maximum speed, and assign that as the speed for the current point.
6. Calculate the angular velocity, using $\omega = v * \kappa$.
7. Increment t using Equation 8.

You need to follow this procedure both forward along the path, and backward. Once you have these two paths, take the minimum of the two at each point, and the result will be a completed motion profile that will follow your path.

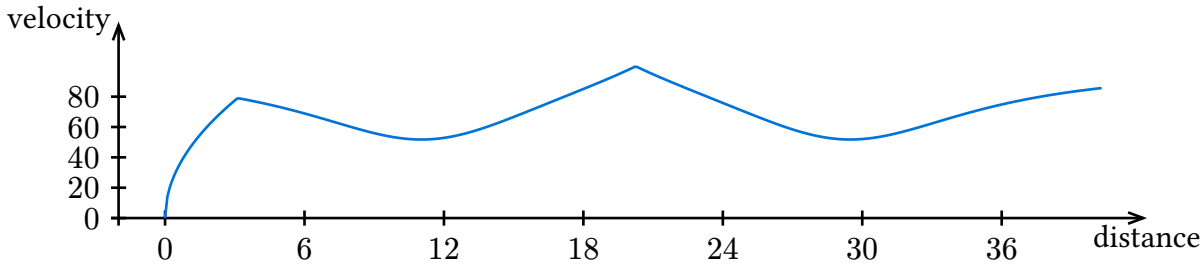
3.3. Example Visualizations

Below are some visualizations that may help to see what a motion profile will look like at different stages of development. Because they are profiled vs distance instead of time, the shapes will look slightly different. All the below examples use the same path as in Equation 7.

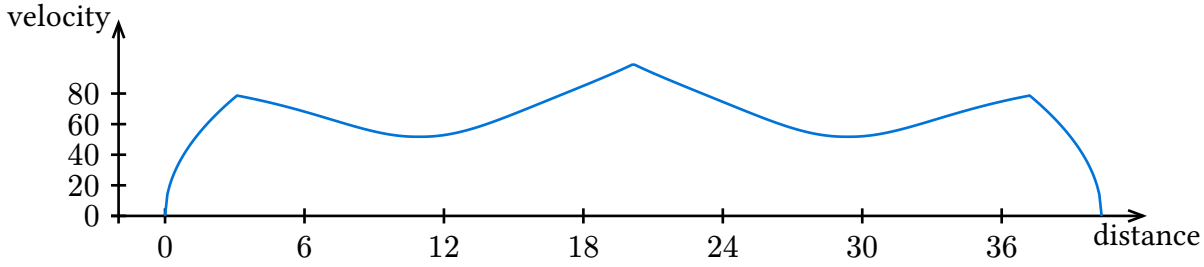
Here is what your first profile might look like, with only velocity limiting and only a forward pass:



This is what it will look like once you add all of the constraints:



This is what it will look like once you add in the proper backward pass:



4. Extending Motion Profiling

There are still some things you will need beyond pure motion profiling to actually control a robot. Here I will briefly cover some of these things, and highlight some areas where this algorithm can be improved.

4.1. Velocity Controller

Once you have a motion profile, you need to actually follow it. Because it outputs only linear and angular velocity, you need to first convert this to wheel velocities, and then command the wheels to move at these velocities.

To calculate individual wheel velocities, you will need to use Equation 11 and Equation 12 to calculate v_l and v_r. You then need to convert these velocities along the ground to rpm, which can be done based on the size of your wheels.

Finally, to convert the desired rpm to a voltage you can apply, you need a velocity controller. You can use the Vex internal velocity controller, but there are many ways to improve upon it. The simplest velocity controller is the feed forward model:

$$V = \mathrm{kS} * \mathrm{sgn}(v) + \mathrm{kV} * v + \mathrm{kA} * a \tag{15}$$

where:
- V is the applied voltage
- v is the target velocity
- a is the target acceleration
- kS is the minimum voltage at which the motor will move
- kV is the velocity gain, which is the voltage the motor need to maintain at a given speed when combined with v
- kA is the acceleration gain, which is the voltage needed to maintain a constant acceleration when combined with a

This is often easiest to tune by looking at a 1D motion profile and adjusting the gains until it is followed very closely.

In addition, you can use a PID to maintain velocity, but I will not go into any more depth on velocity control here.

4.2. Additional Constraints

There are many possible kinematic constraints you could add with this technique, such as limiting angular acceleration, or many others, but one I might suggest is limit speed based on side-to-side friction. This can prevent slipping when taking sharp turns. The formula for maximum speed for a given friction coefficient is:

$$v = \sqrt{\mu * r * g} \tag{16}$$

where:
- μ is the friction coefficient
- r is the radius of the turn, or $\frac{1}{\kappa}$
- g is gravitational acceleration, $9.81 \frac{m}{s^2}$, or $32.174 \frac{\mathrm{ft}}{s^2}$

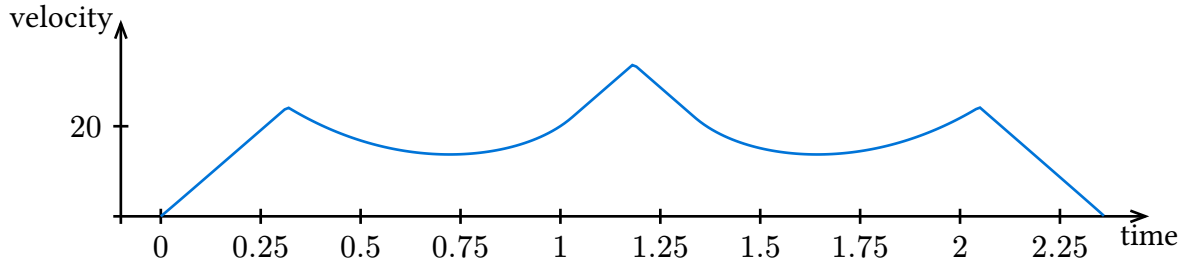
4.3. Time Parameterization

You may want to remap your profile to be velocity vs time at some point, as it can be easier to follow. This is relatively straightforward to do. Simple choose a time interval for spacing, generally 10ms for Vex, and iterate over the distance profile while simulating your current velocity and distance.

The loop might look like:

```
while (currDist < pathDist) {
  currVel = trajectory[(int)(currDist / deltaDist)]
  currDist += currVel * dt;
  timeTrajectory.push_back(currVel);
}
```

This won't be perfect, and I suggest implementing some amount of interpolation, or oversampling to improve the sharpness of the graph. Such a time-parametrized motion profile can be seen below:



4.4. Feedback

Finally, on top of the open loop motion profiling, you will likely want error correction of some sort. The most commonly used algorithm for this is Ramsete[1], which I will not go into in depth, but takes an input error and target, and will output a corrected angular and linear velocity. There are other methods as well, but I will not go into them here, and rather leave it as a suggestion that closing the loop is not necessary, but can allow for much more accurate or quicker movements.

5. Conclusion

I hope you find this document helpful and informative. If you have any further questions, feel free to reach out to me, I am always happy to help people out with this or other programming problems. This is the first document I've written like this, so I am open to any kind of feedback you may have. Please contact me on the Discord as @comodomo, or by email at cbrosgall@gmail.com°.

Bibliography

[1] S. Nicosia, B. Siciliano, A. Bicchi, and P. Valigi, *Ramsete: Articulated and Mobile Robotics for Services and Technologies*. 2001, p. . doi: 10.1007/3-540-45000-9°.

[2] S. Şenyurt and Ş. Kılıçoğlu, "ON THE MATRIX REPRESENTATION OF 5TH ORDER BÉZIER CURVE AND DERIVATIVES IN E 3." [Online]. Available: https://www.researchgate.net/publication/352090603_ON_THE_MATRIX_REPRESENTATION_OF_5TH_ORDER_BEZIER_CURVE_AND_DERIVATIVES_IN_E_3°